



**Mecanismo de detección automática de DERIVA ARQUITECTÓNICA
mediante el uso de Fitness Functions y análisis estático de grafos de
dependencia: Un enfoque de gobernanza continua para PyMES de
software.**

Juan José Medicis Zambrano

Anteproyecto presentado como requisito parcial para optar al título de:
Magister en Ingeniería de Software

Director(a):
Título (Ph.D., MSc) y nombre del director(a)

Pontificia Universidad Javeriana Cali
Facultad de Ingeniería y Ciencias
Departamento de Electrónica y Ciencias de la Computación
Cali, Colombia
27 de marzo de 2026

Ficha Resumen

Anteproyecto de Trabajo de Grado

Posible Título:

1. Área de trabajo:
2. Tipo de proyecto (Aplicado, Innovación, Investigación):
3. Estudiante:
4. Correo electrónico:
5. Dirección y teléfono:
6. Director:
7. Vinculación del director:
8. Correo electrónico del director:
9. Co-Director (Si aplica):
10. Grupo o empresa que lo avala (Si aplica):
11. Otros grupos o empresas:
12. ODS que aplica al proyecto (Agenda 2030): **ver explicación más adelante**
13. Palabras clave(al menos 5):
14. Fecha de inicio:
15. Duración estimada (en meses): cinco meses
16. Resumen: Debe contener el tema a trabajar en el proyecto de grado, su importancia, la problemática que aborda, los objetivos propuestos, resultados esperado y posibles aplicaciones. Debe ser redactado en un solo párrafo y no contener espacios entre líneas ni sangría. Debe ser escrito con máximo 400 palabras

Explicación de los ODS - Borrar esto en el documento final

Los *Objetivos de Desarrollo Sostenible* (ODS) son un conjunto de 17 metas universales adoptadas por la Asamblea General de las Naciones Unidas en 2015 dentro de la Agenda 2030. Se diseñaron para afrontar desafíos globales como la pobreza, la desigualdad, el cambio climático, la degradación ambiental, la paz y la justicia, con el fin de promover un desarrollo equilibrado, inclusivo y sostenible.

En este proyecto, la *alineación con los ODS* implica identificar las metas más relevantes para la problemática central, de modo que los resultados contribuyan directamente al logro de la Agenda 2030. A continuación se presentan los 17 ODS con una breve descripción de cada uno:

1. **ODS 1: Fin de la pobreza.** Erradicar la pobreza en todas sus formas y dimensiones en todo el mundo.
2. **ODS 2: Hambre cero.** Poner fin al hambre, lograr la seguridad alimentaria y mejorar la nutrición, promoviendo la agricultura sostenible.
3. **ODS 3: Salud y bienestar.** Garantizar una vida sana y promover el bienestar para todos en todas las edades.
4. **ODS 4: Educación de calidad.** Garantizar una educación inclusiva, equitativa y de calidad, y promover oportunidades de aprendizaje permanente para todos.
5. **ODS 5: Igualdad de género.** Lograr la igualdad de género y empoderar a todas las mujeres y niñas.
6. **ODS 6: Agua limpia y saneamiento.** Garantizar la disponibilidad y la gestión sostenible del agua y el saneamiento para todos.
7. **ODS 7: Energía asequible y no contaminante.** Asegurar el acceso a energías asequibles, fiables, sostenibles y modernas para todos.

8. **ODS 8: Trabajo decente y crecimiento económico.** Promover un crecimiento económico sostenido, inclusivo y sostenible, el pleno empleo productivo y el trabajo decente para todos.
9. **ODS 9: Industria, innovación e infraestructura.** Construir infraestructuras resilientes, promover la industrialización inclusiva y sostenible, y fomentar la innovación.
10. **ODS 10: Reducción de las desigualdades.** Reducir la desigualdad en y entre los países.
11. **ODS 11: Ciudades y comunidades sostenibles.** Lograr que las ciudades y los asentamientos humanos sean inclusivos, seguros, resilientes y sostenibles.
12. **ODS 12: Producción y consumo responsables.** Garantizar modalidades de consumo y producción sostenibles.
13. **ODS 13: Acción por el clima.** Adoptar medidas urgentes para combatir el cambio climático y sus efectos.
14. **ODS 14: Vida submarina.** Conservar y utilizar en forma sostenible los océanos, los mares y los recursos marinos.
15. **ODS 15: Vida de ecosistemas terrestres.** Gestionar sosteniblemente los bosques, luchar contra la desertificación, detener y revertir la degradación de la tierra y detener la pérdida de biodiversidad.
16. **ODS 16: Paz, justicia e instituciones sólidas.** Promover sociedades pacíficas e inclusivas, garantizar el acceso a la justicia para todos y construir instituciones eficaces, responsables e inclusivas.
17. **ODS 17: Alianzas para lograr los objetivos.** Revitalizar la Alianza Mundial para el Desarrollo Sostenible y fortalecer los medios de implementación.

Índice

1. Planteamiento y Justificación del Problema	7
1.1. Planteamiento del Problema:	7
1.2. Justificación del Proyecto:	8
2. Objetivos del proyecto	8
2.1. Objetivo General	8
2.2. Objetivos específicos	8
3. Alcance del Proyecto	9
4. Marco teórico y antecedentes	10
4.1. Bases Teóricas	10
4.2. Antecedentes	12
5. Riesgos Éticos	14
6. Metodología del proyecto	15
6.1. Fases del Trabajo	16
6.2. Justificación de las Decisiones Metodológicas	17
7. Cronograma	17

Índice de figuras

Índice de tablas

1. Desglose de 34 h del Mes 1 (incluye 10 % holgura)	19
--	----

Todo list

1. Planteamiento y Justificación del Problema

1.1. Planteamiento del Problema:

En el ecosistema actual de desarrollo de software, la arquitectura ha dejado de ser un plano estático para convertirse en una entidad dinámica que evoluciona en cada iteración. Este dinamismo da origen a la deriva arquitectónica (architecture drift), definida como la divergencia progresiva entre la arquitectura planeada (prescrita) y la arquitectura realmente implementada en el código fuente (actual) (Martín y col., s.f.).

La investigación indica que aproximadamente el 65 % del presupuesto de TI de una organización se consume exclusivamente en el mantenimiento de software, a menudo debido a problemas inducidos por la erosión arquitectónica. Esta erosión puede incrementar los costos de desarrollo entre un 20 % y 40 % en comparación con sistemas bien mantenidos. Sin embargo, el 80 % de las organizaciones ágiles y PyMES carecen de documentación arquitectónica actualizada, lo que invalida los métodos tradicionales de gobernanza basados en revisiones manuales o comités (Manglaras y col., 2026).

Sin mecanismos automatizados, la deriva introduce "smells" estructurales como dependencias cíclicas, componentes "Dios" (God Components) y violaciones de capas, que elevan el Índice de Deuda Arquitectónica (ADI) ("Transforming enterprise architecture into a decision infrastructure – Djimit van data naar doen." s.f.). Esto lleva al "Legacy Trap", donde la complejidad acumulada impide la innovación estratégica ("Transforming enterprise architecture into a decision infrastructure – Djimit van data naar doen." s.f.). Por tanto, existe una necesidad crítica de mecanismos que realicen una gobernanza continua basándose exclusivamente en el código fuente, transformando las reglas de diseño en Fitness Functions ejecutables ("Make your architecture agile with fitness functions | Okoone", s.f.).

1.2. Justificación del Proyecto:

En ausencia de un mecanismo automatizado de detección de deriva arquitectónica, las PyMES de software continúan operando bajo un modelo reactivo de gobernanza técnica, donde la arquitectura solo se revisa cuando ya existen fallos visibles en el sistema. Esta falta de visibilidad estructural genera una acumulación progresiva de desviaciones entre la arquitectura diseñada y la implementada, fenómeno que no se manifiesta inicialmente como errores funcionales, sino como deterioro silencioso de la calidad interna del sistema (Jolak y col., 2025).

El proyecto propuesto no busca prevenir fallos visibles, sino evitar que el software se convierta en un sistema rígido incapaz de adaptarse al cambio — una condición que representa uno de los principales riesgos estratégicos para las PyMES en entornos digitales altamente dinámicos.

2. Objetivos del proyecto

2.1. Objetivo General

Desarrollar un mecanismo automatizado para la detección de derivas arquitectónicas, basado en grafos de dependencia y la ejecución de funciones de fitness, eliminando la dependencia de documentación externa para garantizar la integridad estructural en proyectos de software de PyMES.

2.2. Objetivos específicos

1. Definir un conjunto de Fitness Functions basadas en el análisis de smells arquitectónicos más comunes (acoplamiento eferente, ciclos y violación de capas) adaptadas a entornos de desarrollo ágil.
2. Implementar un componente de análisis estático que genere automáticamente grafos de dependencia y topologías de componentes a partir del código fuente y de archivos de configuración, usando una herramienta propietaria.

3. Construir un motor de validación que compare el grafo extraído con las reglas definidas, calculando una **Tasa de Conformidad (CR)** automatizada en ciertos aspectos definidos.
4. Evaluar el mecanismo en un proyecto en condiciones controladas, analizando su capacidad para identificar los smells arquitectónicos comunes y determinar su efectividad en proyectos de software de PyMES.

3. Alcance del Proyecto

- **Dominio Tecnológico:** El prototipo se limitará a un único ecosistema (Java o .NET) para garantizar la profundidad técnica dentro del tiempo establecido..
- **Reglas de Detección:** El análisis se enfocará en tres dimensiones: integridad de capas (ej. la capa de persistencia no debe llamar a la de presentación), ausencia de ciclos de dependencia y cumplimiento de convenciones de nomenclatura arquitectónica (Niessen, s.f.).
- **Entregables:** Un repositorio con el prototipo funcional, un catálogo de reglas pre-configuradas y el informe final de investigación.

4. Marco teórico y antecedentes

4.1. Bases Teóricas

Esta sección fundamenta los conceptos pilares sobre los cuales se construye el mecanismo de detección automatizada.

- **Deriva y Erosión Arquitectónica:** Se define como la divergencia progresiva entre la arquitectura prescrita (planeada) y la arquitectura realmente implementada en el código fuente. Este fenómeno implica una pérdida de integridad estructural debido a la violación de principios de diseño originales (Pigazzini, 2021). La investigación reciente destaca que esta divergencia es un problema persistente cuando no existen acciones de contención, impactando negativamente en la mantenibilidad y la capacidad de evolución del sistema (Anthony y col., 2024). Se estima que el costo del mantenimiento de software, a menudo inflado por la erosión, consume aproximadamente el 65 % del presupuesto de TI de una organización (Pigazzini, 2021).
- **Deuda Técnica Arquitectónica (ATD):** Es una metáfora que describe las decisiones de diseño subóptimas que ofrecen beneficios a corto plazo (como entregas rápidas) pero deterioran la calidad a largo plazo, generando un interés.^{en} forma de mayor esfuerzo de mantenimiento (Hamed y col., 2023). La ATD se diferencia de la deuda a nivel de código porque involucra decisiones estructurales globales como capas, subsistemas e interfaces (Pigazzini, 2021). Un síntoma clave de su acumulación es la presencia de Smells Arquitectónicos (AS) (Sas & Avgeriou, 2022).
- **Smells Arquitectónicos (AS):** Son decisiones de diseño que impactan negativamente en la calidad interna del sistema (Pigazzini, 2021). A diferencia de los code smells, que ocurren a nivel de métodos o clases, los AS involucran múltiples componentes, paquetes o capas (Sas & Avgeriou, 2022). Ejemplos críticos incluyen la **Dependencia Cíclica (CD)**, donde los componentes no pueden ser liberados o probados de forma aislada, y el **Componente Dios (GC)**,

que ocurre cuando una entidad es excesivamente grande y difícil de entender (Pigazzini, 2021; Sas & Avgeriou, 2022).

- **Funciones de Aptitud (Fitness Functions):** Una función de aptitud es un mecanismo diseñado para resumir qué tan cerca está una solución de diseño específica de alcanzar sus objetivos y preservar sus características clave. Se definen formalmente como "cualquier mecanismo que proporciona una evaluación de integridad objetiva de alguna característica arquitectónica". Su propósito fundamental es actuar como barandillas (guardrails) automatizadas que comunican, validan y preservan atributos de calidad (requisitos no funcionales) de manera continua, asegurando que la evolución del sistema se mantenga dentro de un rango y una dirección deseados por los arquitectos. Esta disciplina permite a los equipos ser reactivos ante la degradación de características críticas en tiempo real, evitando que la realidad del sistema en producción se desvíe de las intenciones de diseño originales ("Fitness Functions – Safeguard Architecture with Automated Checks", s.f.).
- **Análisis Estático y Grafos de Dependencia:** Es la técnica de extraer hechos y abstracciones del código fuente sin ejecutarlo (Pigazzini, 2021). Los grafos de dependencia representan los componentes del sistema como nodos y sus relaciones como aristas, permitiendo aplicar algoritmos de búsqueda (como *Depth First Search*) para detectar estructuras problemáticas como ciclos (Klein y col., 2022; Pigazzini, 2021).
- **Algoritmos de Recorrido sobre el Grafo de Dependencias:** La detección de smells arquitectónicos sobre el grafo $G = (V, E)$ requiere la aplicación de algoritmos de grafos dirigidos. La **Dependencia Cíclica (CD)** se detecta mediante una búsqueda en profundidad (*DFS*) que identifica Componentes Fuertemente Conexos (SCC), clasificando un subconjunto de nodos como cíclico cuando $|SCC| > 1$. La **Dependencia Inestable (UD)** se calcula midiendo el acoplamiento aferente (C_a = aristas entrantes) y eferente (C_e = aristas salientes) de cada nodo, derivando la métrica $I = C_e / (C_a + C_e)$; una relación

$A \rightarrow B$ constituye un UD cuando el componente estable A depende del inestable B ($I_A < I_B$). El **Componente Dios (GC)** se detecta cuando el número de dependencias de salida de un paquete supera un umbral configurable. Sas y col. (2019) validaron empíricamente estas métricas analizando la evolución de smells de inestabilidad en proyectos Java de código abierto mediante el recorrido sistemático del grafo de dependencias a nivel de paquete (Sas y col., 2019).

4.2. Antecedentes

La investigación actual presenta diversas estrategias para gestionar la integridad técnica del software, evolucionando desde la medición manual hacia enfoques predictivos y automatizados.

- **ArchUnit:** Es una biblioteca gratuita y extensible diseñada para verificar la arquitectura de código Java mediante cualquier entorno de pruebas unitarias estándar. Funciona analizando el bytecode e importando las clases en una estructura de código Java, lo que permite comprobar dependencias entre paquetes, capas, ciclos y más. Es la herramienta que permite materializar el concepto de "arquitectura auto-testeable" al permitir escribir reglas arquitectónicas como si fueran pruebas unitarias comunes ("Unit test your Java architecture - ArchUnit", s.f.).
- **ArchSync:** Este es una herramienta de asistencia que busca mantener en sincronía el código con los escenarios arquitectónicos (Use Case Maps). ¿Qué hace?, Utiliza heurísticas para procesar trazas de ejecución y correlacionarlas con el comportamiento arquitectónico esperado, sugiriendo "scripts de actualización" para corregir inconsistencias. Aunque ArchSync se apoya más en el comportamiento dinámico (trazas), coincide en reducir el esfuerzo de sincronización manual entre arquitectos y desarrolladores. (Diaz-Pace y col., 2011)
- **Gestión en PyMES:** (Hamed y col., 2023) proponen un enfoque de gestión de deuda técnica diseñado específicamente para pequeñas y medianas empresas

(SMEs). Destacan que las limitaciones de recursos, presupuestos ajustados y la falta de documentación actualizada en estos entornos hacen vital una gobernanza basada en herramientas de análisis estático (como SonarQube) que operen sobre el artefacto vivo del software (Hamed y col., 2023).

- **Índices de Deuda Basados en Machine Learning:** (Sas & Avgeriou, 2022) desarrollaron el **ATDI (Architectural Technical Debt Index)**, el cual utiliza modelos de aprendizaje automático (gradient boosting) para clasificar la severidad de los smells arquitectónicos. Este enfoque combina el análisis estático de las líneas de código afectadas con la severidad estimada, logrando un 71 % de acuerdo con la percepción de esfuerzo de refactorización de desarrolladores expertos (Sas & Avgeriou, 2022).
- **Conformidad Automatizada en el Flujo de CI/CD:** (Klein y col., 2022) subrayan que el uso de verificadores de conformidad automatizados integrados en los flujos de Integración Continua (CI) permite exponer las desviaciones en el momento del commit. Esto facilita la remediación inmediata antes de que las violaciones se fijen en la implementación, evitando que la deriva se convierta en un problema de mantenimiento costoso meses o años después (Klein y col., 2022).
- **Modelo de Reflexión (Software Reflexion Models):** Murphy y col. (2001) introdujeron el mecanismo formal para cuantificar la divergencia entre el grafo de dependencias del código fuente (*source model*) y la arquitectura prescrita (*high-level model*), clasificando cada relación en convergencias (permitidas), divergencias (violaciones = deriva) o ausencias (esperadas pero faltantes). Esta tripartición fundamenta directamente el motor de validación y la Tasa de Conformidad (CR) de este trabajo (Murphy y col., 2001).

5. Riesgos Éticos

El desarrollo de un mecanismo automatizado para la gobernanza técnica en PyMES conlleva implicaciones éticas centradas en la privacidad, la equidad algorítmica y el impacto sociotécnico en los equipos de desarrollo.

- **Protección de información sensible:** El análisis estático requiere acceso directo al código fuente, el cual constituye el activo más valioso de una PyME. Existe el riesgo de exposición de lógica de negocio o secretos industriales si los grafos de dependencia no se manejan bajo protocolos de confidencialidad estrictos (Hamed y col., 2023; Modawski & Wolniak, 2025).
- **Sesgo algorítmico y falta de transparencia:** Los modelos de aprendizaje automático utilizados para calcular índices como el ATDI pueden presentar sesgos si los datos de entrenamiento no son representativos o si se basan en umbrales arbitrarios (Sas & Avgeriou, 2022). Un diagnóstico erróneo de la severidad de un *smell* podría llevar a decisiones de refactorización costosas e innecesarias (Sas & Avgeriou, 2022).
- **Impacto social y moral del equipo:** La deuda técnica tiene un impacto documentado y negativo en la moral de los desarrolladores (Hamed y col., 2023). La automatización de la gobernanza corre el riesgo de ser percibida como una herramienta de vigilancia punitiva en lugar de un soporte técnico, lo que podría aumentar el estrés laboral si los resultados se utilizan para evaluaciones de desempeño individual sin considerar el contexto del negocio (Hamed y col., 2023; Paul & Wang, 2019).
- **Riesgo de formalismo vacío:** Existe la posibilidad de que los equipos traten las *Fitness Functions* como un simple trámite burocrático (marcar casillas), ignorando la evaluación crítica del diseño por cumplir con una métrica de conformidad (CR) automatizada (Modawski & Wolniak, 2025).
- **Veracidad del diagnóstico y alcance parametrizable de la herramienta:** La Tasa de Conformidad (CR) producida por el mecanismo es un indicador

circunscrito exclusivamente a las reglas configuradas en el prototipo, las cuales se limitan a las tres dimensiones explicitadas en el alcance del proyecto: integridad de capas, ausencia de ciclos de dependencia y cumplimiento de convenciones de nomenclatura. El riesgo ético consiste en que directivos, auditores o equipos de desarrollo interpreten una CR elevada como sinónimo de arquitectura completa y correctamente gobernada, cuando en realidad solo certifica el cumplimiento de las reglas programadas sobre el ecosistema evaluado. Esta confusión puede generar una falsa sensación de control, inhibir revisiones manuales complementarias y llevar a decisiones de inversión o despliegue basadas en información parcial. Para mitigar este riesgo, el prototipo deberá parametrizar su alcance de forma transparente: cada reporte debe declarar explícitamente qué reglas se evaluaron, sobre qué módulos y con qué umbrales, acompañado de una advertencia en los resultados que indique que el diagnóstico **no cubre** aspectos arquitectónicos fuera del alcance declarado. Asimismo, la herramienta no debe presentarse como un árbitro definitivo de la calidad arquitectónica, sino como un mecanismo de detección temprana acotado a su dominio de configuración (Klein y col., 2022; Modawski & Wolniak, 2025).

6. Metodología del proyecto

La estrategia general de este proyecto se fundamenta en la metodología **Design Science Research (DSR)**, la cual permite la creación y evaluación de artefactos tecnológicos diseñados para resolver problemas prácticos identificados en la ingeniería de software (Hamed y col., 2023). En este contexto, el problema abordado es la falta de gobernanza técnica continua en PyMES, y el artefacto propuesto es un mecanismo automatizado de detección de deriva arquitectónica.

El enfoque del trabajo es eminentemente técnico y se divide en cuatro fases iterativas, alineadas directamente con los objetivos específicos planteados.

6.1. Fases del Trabajo

1. **Fase de Definición y Especificación (Objetivo 1):** Se procederá a la formalización de las reglas de diseño bajo el paradigma de la arquitectura evolutiva. Utilizando el concepto de *Fitness Functions* (Paul & Wang, 2019), se traducirán principios abstractos (como la integridad de capas y la modularidad) en indicadores métricos objetivos y ejecutables. Se priorizarán métricas de acoplamiento eferente y detección de ciclos por ser síntomas críticos de erosión (Pigazzini, 2021).
2. **Fase de Implementación del Analizador Estático (Objetivo 2):** Se desarrollará un componente capaz de extraer hechos arquitectónicos directamente del código fuente Java sin requerir ejecución. Siguiendo modelos de inferencia de diseño (Klein y col., 2022), el componente procesará archivos .java y configuraciones de construcción (Maven/Gradle). Los datos extraídos se representarán mediante **grafos de dependencia**, donde las clases y paquetes actuarán como nodos y sus relaciones como aristas, utilizando bases de datos orientadas a grafos para facilitar consultas estructurales complejas (Klein y col., 2022; Pigazzini, 2021).
3. **Fase de Construcción del Motor de Validación (Objetivo 3):** Se programará el motor lógico que contrastará el grafo de la arquitectura implementada (extraído en la fase anterior) contra el modelo de la arquitectura prescrita definido por las *Fitness Functions*. El resultado principal será el cálculo automático de la **Tasa de Conformidad (CR)**, que proporcionará una visión cuantitativa de la integridad del sistema (Paul & Wang, 2019).
4. **Fase de Evaluación mediante Estudio de Caso (Objetivo 4):** El mecanismo se validará en un entorno controlado (real o simulado) con un esfuerzo de 192 horas de investigación. La técnica de recolección de datos consistirá en la **inyección voluntaria de derivas arquitectónicas** (violación de capas y ciclos de dependencia) para medir la sensibilidad y precisión del motor en su detección. Se utilizarán índices de referencia como el *Architectural Tech-*

nical Debt Index (ATDI) para contrastar la efectividad del prototipo (Sas & Avgeriou, 2022).

6.2. Justificación de las Decisiones Metodológicas

La elección de **Design Science Research** se justifica por su capacidad para cerrar la brecha entre la teoría académica y la práctica industrial en PyMES, donde la documentación suele ser inexistente o estar desactualizada (Anthony y col., 2024; Hamed y col., 2023).

El uso de **análisis estático** basado en el código como "única fuente de verdad" es una decisión crítica para garantizar que la gobernanza sea viable en entornos ágiles, evitando la dependencia de artefactos externos que introducen errores de interpretación (Anthony y col., 2024; Klein y col., 2022). Finalmente, la adopción de **Fitness Functions** permite integrar la validación arquitectónica directamente en el flujo de desarrollo, transformando la gobernanza técnica de una revisión manual esporádica a un proceso de monitoreo continuo y objetivo (Niessen, s.f.; Paul & Wang, 2019).

7. Cronograma

Extensión máxima: 2 páginas

Presente un cronograma realista para el desarrollo del proyecto en 5 meses (192 horas). Esto implica una dedicación aproximada de 9 horas a la semana.

- Divida el trabajo en fases o entregables mensuales.
- Incluya actividades clave alineadas con los objetivos.
- Puede usarse una tabla o diagrama de Gantt.

Le recomendamos seguir esta guía:

1. **Seleccione y adapte una metodología para el desarrollo del proyecto.**Ejemplo

- *En cascada*: fases secuenciales (requisitos → diseño → ejecución → verificación → cierre).
 - *Iterativa*: ciclos cortos que repasan análisis, diseño y pruebas varias veces.
 - *Ágil/Scrum*: divide en sprints de 2–4 semanas con entregables incrementales.
2. **Defina fases o sprints.** A cada fase o sprint asígnese un objetivo y un entregable claro.
 3. **Estime horas por fase.** Reparta las 192 h entre fases en proporción al esfuerzo previsto. P. ej.:
 - Investigación y planificación: 10 % (aprox 19 h)
 - Ejecución principal (diseño/prototipo/experimento): 50 % (aprox 96 h)
 - Evaluación y ajustes: 15 % (aprox 29 h)
 - Reuniones con el director, documentación y defensa: 25 % (aprox 48 h)
 4. **Distribuya semanalmente.** Con base en la estimación por fase, calcule horas/semana y asigne las tareas de 4–10 h cada una
 5. **Desglose en tareas concretas.** Para cada fase/sprint, liste actividades (p. ej.: lectura de artículos, diseño de experimento, programación de prototipo, análisis de datos, redacción de capítulos) y señale:
 - Fecha de inicio
 - Duración estimada (h)

Reserve 20 h para imprevistos o refinamientos.
 6. **Establezca controles periódicos.** Agende revisiones cada 4–6 semanas (o al cierre de cada sprint) para:
 - Comparar horas planificadas vs. reales

- Ajustar el cronograma
- Recoger feedback de director - codirector

7. **Visualice y monitorice.** Use diagrama de Gantt, tablero Kanban o tabla de seguimiento en hoja de cálculo.

Ejemplo de desglose semanal (primer mes, 9 h/semana):

Semana	Actividad	Horas	Saldo Mes
1	Revisión bibliográfica y marco teórico	9	29
2	Diseño de cuestionario / experimento inicial	9	20
3	Revisión siguiendo y metodología con el director	8	12
4	Ajustes y planificación detallada	8	4

Tabla 1: Desglose de 34 h del Mes 1 (incluye 10 % holgura)

Consejos finales:

- *Transparencia:* Documente cambios (qué, por qué, cuándo). Esto le será muy útil para escribir su documento final.
- *Flexibilidad:* Recalibre fases según el progreso real.
- *Comunicación:* Comparta avances regulares con el director, normalmente los avances semanales permiten mejor avance en los proyectos. Si tiene algun problema de comunicación con el director durante la ejecución del proyecto, repórtelo a la dirección del programa para buscar en conjunto soluciones.

Se deben presentar de forma rigurosa y completa las referencias bibliográficas utilizadas en el documento (No incluir bibliografía no referenciada en el documento). Se debe seguir una sola norma de referenciación en todo el documento. Para este proyecto se solicita APA. Tenga en cuenta que por la plantilla a utilizar las referencias contextuales se citan con **citet**.

Por ejemplo (**DaSilvaCosta2022**) dijo xx y z...

Las referencias parentéticas se citan con el comando **citep**.

Por ejemplo, esto es lo que quiero presentar (**Glorot2011**)

NO use el comando cite porque no funciona bien en esta plantilla

Utilice en lo posible bibliografía reciente de fuentes confiables y en inglés (libros, artículos científicos, etc.). Evitar utilizar fuentes no confiables como blogs, Wikipedia, o documentos sin autor.

Referencias

- Anthony, E., Berntsson, A., Santilli, T. & Wohlrab, R. (2024). We're Drifting Apart: Architectural Drift from the Developers' Perspective. *Proceedings of the IEEE 21st International Conference on Software Architecture (ICSA 2024)*, 101-111. <https://doi.org/10.1109/ICSA59870.2024.00018>
- Diaz-Pace, J. A., Soria, A., Rodriguez, G. & Campo, M. R. (2011). Taking ArchSync to the Real World: An Analysis of Three Case-Studies. Fitness Functions – Safeguard Architecture with Automated Checks. (s.f.). <https://continuous-architecture.org/practices/fitness-functions/>
- Hamed, A. S. E., Elbakry, H. M., Riad, A. E. & Moawad, R. (2023). A Proposed Technical Debt Management Approach Applied on Software Projects in Egypt. *Journal of Internet Services and Information Security (JISIS)*, 13(3), 156-177. <https://doi.org/10.58346/JISIS.2023.I3.010>
- Jolak, R., Karlsson, S. & Dobslaw, F. (2025). An empirical investigation of the impact of architectural smells on software maintainability. *Journal of Systems and Software*, 225, 112382. <https://doi.org/10.1016/j.jss.2025.112382>

- Klein, J., Nord, R. & Shull, F. (2022). *Why Architecture Conformance Matters for Software Systems* (inf. téc. N.º DM22-0924). Software Engineering Institute, Carnegie Mellon University.
- Make your architecture agile with fitness functions | Okoone. (s.f.). <https://www.okoone.com/spark/technology-innovation/make-your-architecture-agile-with-fitness-functions/>
- Manglaras, O., Farkas, A., Woolford, T., Treude, C. & Wagner, M. (2026). Mo-dARO: A Modular Approach to Architecture Reconstruction of Distributed Microservice Codebases. <https://arxiv.org/pdf/2602.08181v1>
- Martín, D. S., Camargo, V. V. D. & Angulo, G. (s.f.). Architectural Conformance Checking for MAPE-K-based Self-Adaptive Systems.
- Modawski, A. & Wolniak, A. (2025). Lightweight AI Governance (LAIG) Framework for SMEs. *Proceedings of the European Workshop on Trustworthy AI (TRUST-AI 2025), organized as part of the European Conference of Artificial Intelligence (ECAI 2025)*. <https://ceur-ws.org/Vol-4132/short57.pdf>
- Murphy, G. C., Notkin, D. & Sullivan, K. J. (2001). Software Reflexion Models: Bridging the Gap between Source and High-Level Models. *IEEE Transactions on Software Engineering*, 27(4), 364-380. <https://doi.org/10.1109/32.908994>
- Niessen, L. (s.f.). Fitness Functions: Automating Your Architecture Decisions | by Lukas Niessen | Feb, 2026 | Medium. <https://lukasniessen.medium.com/fitness-functions-automating-your-architecture-decisions-08b2fe4e5f34>
- Paul, P. & Wang, R. (2019). Fitness function-driven development. *Thoughtworks*. <https://www.thoughtworks.com/insights/articles/fitness-function-driven-development>
- Pigazzini, I. (2021). *Evaluating and Detecting Architecture Erosion* (PhD Thesis). Università degli Studi di Milano-Bicocca.
- Sas, D. & Avgeriou, P. (2022). An architectural technical debt index based on machine learning and architectural smells [Bernoulli Institute, University of Groningen].
- Sas, D., Avgeriou, P. & Fontana, F. A. (2019). Investigating Instability Architectural Smells Evolution: An Exploratory Case Study. *2019 IEEE International*

Conference on Software Maintenance and Evolution (ICSME), 557-567. <https://doi.org/10.1109/ICSME.2019.00090>

Transforming enterprise architecture into a decision infrastructure – Djimit van data naar doen. (s.f.). <https://djimit.nl/transforming-enterprise-architecture-into-a-decision-infrastructure/>

Unit test your Java architecture - ArchUnit. (s.f.). <https://www.archunit.org/>